Department of Electronic & Telecommunication Engineering
University of Moratuwa, Sri Lanka

IEEE 754 Single Precision Floating Point Unit

Submitted for partial fulfillment of the EN3021 – Digital System Design.

Name: Weragoda W.A.A.P
Index: 220692R

Date: December 28, 2025

Contents

1 Introduction **2**
1.1 IEEE 754 Single Precision Standard 2
1.2 System Interface 2

2 Architecture Overview **3**
2.1 Clock Divider (`clkdiv.v`) 3
2.2 Floating Point Multiplier (`fpMul.v`) 3
2.3 Floating Point Divider (`divider.v`) 4
2.4 5. Exponent ALU (Adder/Subtractor) (`exponentAlu.v`) 4
2.5 Normalizer and Rounding (`normalizer.v`) 5

3 Simulation and Verification **5**
3.1 Functional Simulation 5
3.2 Timing Analysis 7
3.3 Resource Utilization 7

4 Conclusion **8**

1 Introduction

The Floating Point Unit (FPU) is a fundamental component for performing arithmetic on real numbers spanning a wide dynamic range. This project implements a **Sequential Finite State Machine (FSM) based FPU** targeting the IEEE 754 Single Precision standard (32-bit).

Unlike pipelined architectures that optimize for throughput, this design optimizes for resource efficiency and control simplicity. It utilizes a central controller to dispatch operations to specialized sub-modules (Adder, Multiplier, Divider) and ensures correct handling of floating-point exceptions such as Overflow, Underflow, and Division by Zero.

1.1 IEEE 754 Single Precision Standard

The design adheres to the IEEE 754-2008 standard for 32-bit single-precision floating-point arithmetic. This format allows for a dynamic range of approximately 10^{-38} to 10^{38}. The 32-bit word is divided into three distinct fields:

- **Sign (S):** 1 bit [Bit 31]. Indicates positive (0) or negative (1).
- **Exponent (E):** 8 bits [Bits 30–23]. Represents the magnitude using an excess-127 bias ($Bias = 127$).
- **Mantissa (M):** 23 bits [Bits 22–0]. Represents the fractional precision. For normalized numbers, there is an implicit leading '1' to the left of the binary point (i.e., $1.M$).

The numerical value V of a stored floating-point number is calculated as:

$$V = (-1)^S \times (1.M) \times 2^{E-127} \tag{1}$$

The architecture explicitly handles special cases defined by the standard:

- **Zero:** $E = 0, M = 0$.
- **Infinity:** $E = 255, M = 0$ (Used to flag Overflow or Division by Zero).
- **NaN (Not a Number):** $E = 255, M \neq 0$ (Used for invalid operations like $\frac{0}{0}$).
- **Denormalized Numbers:** $E = 0, M \neq 0$ (Handled by the decoder as $2^{-126} \times 0.M$).

1.2 System Interface

The FPU interacts with the host system via a memory-mapped or UART-bridge interface (handled by the `core` module).

- **Inputs:** Operand A (32-bit), Operand B (32-bit), Opcode (3-bit), Enable Signal.
- **Outputs:** Result C (32-bit), Busy Flag, Exception Flags, Condition Codes.

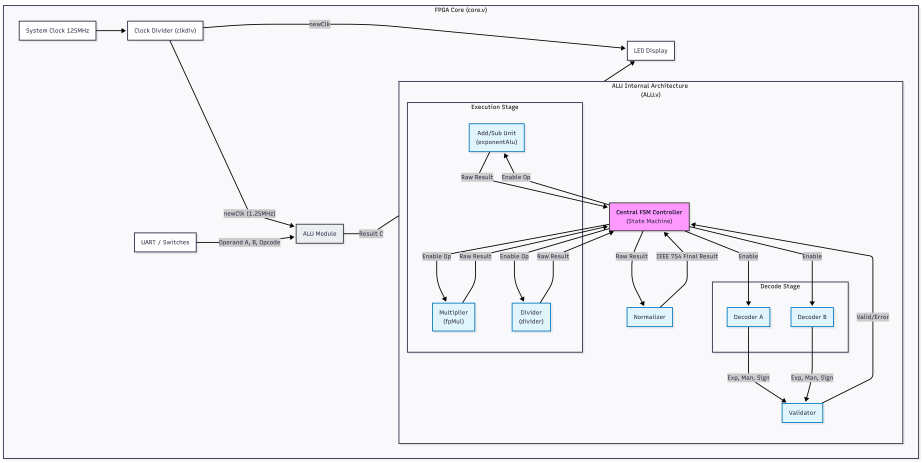

Figure 1: Top-Level System Architecture

2 Architecture Overview

2.1 Clock Divider (`clkdiv.v`)

The system utilizes a dual-clock architecture. The primary `clk` operates at 125 MHz, while the arithmetic core runs on a derived `newClk` at 1.25 MHz. This frequency scaling is essential for achieving timing closure on the long carry chains present in the divider logic.

2.2 Floating Point Multiplier (`fpMul.v`)

The multiplier module implements a **Sequential Shift-and-Add Algorithm**. Unlike combinational multipliers that consume significant DSP resources and have long propagation delays, this sequential approach calculates the 48-bit product over 24 clock cycles.

Algorithmic Process:

1. **Initialization:** The multiplicand (Mantissa A) is loaded into a register. The multiplier (Mantissa B) is loaded into the temporary accumulator (`temp`).
2. **Iterative Calculation (24 Cycles):**
   - The logic inspects the Least Significant Bit (LSB) of `temp`.
   - **If LSB is 1:** The multiplicand (shifted left by 24 bits) is added to the upper half of the accumulator.
   - **Shift:** The entire accumulator is right-shifted by 1 bit. This aligns the partial sum for the next iteration and moves the next bit of the multiplier into the LSB position.
3. **Exponent Calculation:** The new exponent is computed in parallel: $E_{res} = E_A + E_B - 127$ (Bias).

2.3 Floating Point Divider (`divider.v`)

The divider is the most computationally intensive module. It implements a **Restoring Division Algorithm** (Digit Recurrence), which performs division bit-by-bit, similar to manual long division.

Algorithmic Process:

1. **Setup:**
   - The Dividend is padded with zeros to create a 75-bit register `q`.
   - The Divisor's 2's complement is pre-calculated (`neg`) to allow subtraction via addition.
2. **Division Loop (75 Cycles):**
   - **Shift:** The Remainder (`r`) is left-shifted to pull in the MSB from the Dividend (`q`).
   - **Trial Subtraction:** The Divisor is subtracted from the Remainder ($r + neg$).
   - **Decision:**
     - If the result is negative ($MSB = 1$), the divisor was too large. The Remainder is *restored* by adding the divisor back, and the quotient bit is set to 0.
     - If the result is positive, the subtraction stands, and the quotient bit is set to 1.
3. **Normalization Optimization:** The algorithm dynamically detects the position of the first significant bit during calculation, passing a `firstReg` value to the normalizer to skip redundant shift cycles.

2.4 5. Exponent ALU (Adder/Subtractor) (`exponentAlu.v`)

The `exponentAlu` module performs floating-point addition and subtraction. Unlike integer arithmetic, floating-point addition requires the exponents of both operands to be identical before the mantissas can be added. This unit handles exponent comparison, mantissa alignment (barrel shifting), and the core arithmetic operation using 2's complement logic.

Algorithmic Process:

1. **Exponent Comparison:** The difference between the two exponents is calculated: $\delta = E_A - E_B$.
2. **Alignment (Shift):** The mantissa of the number with the smaller exponent is right-shifted by $|\delta|$ bits. This "denormalizes" the smaller number so its bit-weight matches the larger number.
3. **2's Complement Conversion:** To handle both addition and subtraction (and negative inputs) using a single adder structure, the mantissas are converted to signed 2's complement format based on their sign bits and the requested operation.
4. **Arithmetic & Sign Repair:** The aligned signed mantissas are added. If the result is negative, it is converted back to sign-magnitude form for the normalization stage.

2.5 Normalizer and Rounding (`normalizer.v`)

The normalizer post-processes the raw results from the arithmetic units. It ensures the output strictly adheres to the IEEE 754 format.

Key Functions:

- **Priority Encoder:** Scans the intermediate result to locate the leading '1'.
- **Alignment:** Left-shifts the mantissa to place the leading '1' at the hidden bit position (bit 23).
- **Rounding:** Implements the IEEE 754 *Round to Nearest, Ties to Even* mode using Guard, Round, and Sticky bits. This logic handles edge cases where rounding up causes a mantissa overflow (e.g., $1.11...1 \rightarrow 10.00...0$), requiring a subsequent exponent increment.

3 Simulation and Verification

3.1 Functional Simulation

The design was verified using a Verilog testbench (`tb_ALU.v`). Test cases covered:

- Normal Addition/Subtraction (Checking exponent alignment).
- Multiplication (Verifying 48-bit product truncation).
- Corner Cases: Adding large and small numbers (swamping), cancellation of close numbers.

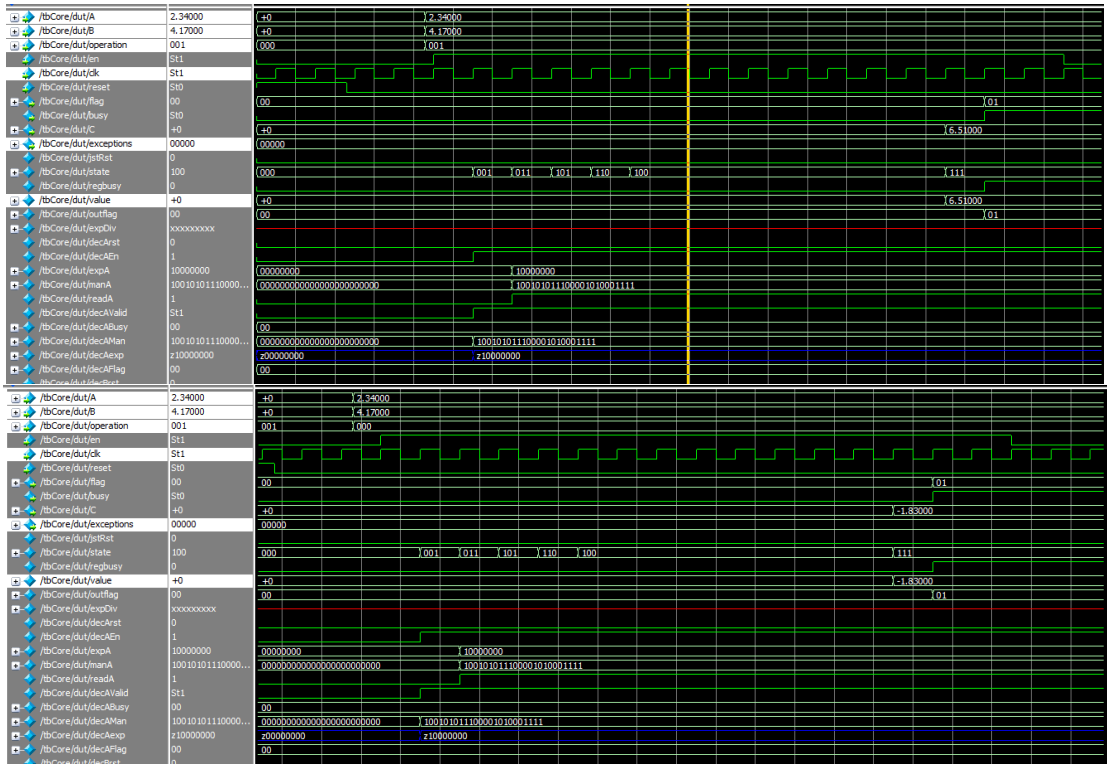

Figure 2: Addition and substraction

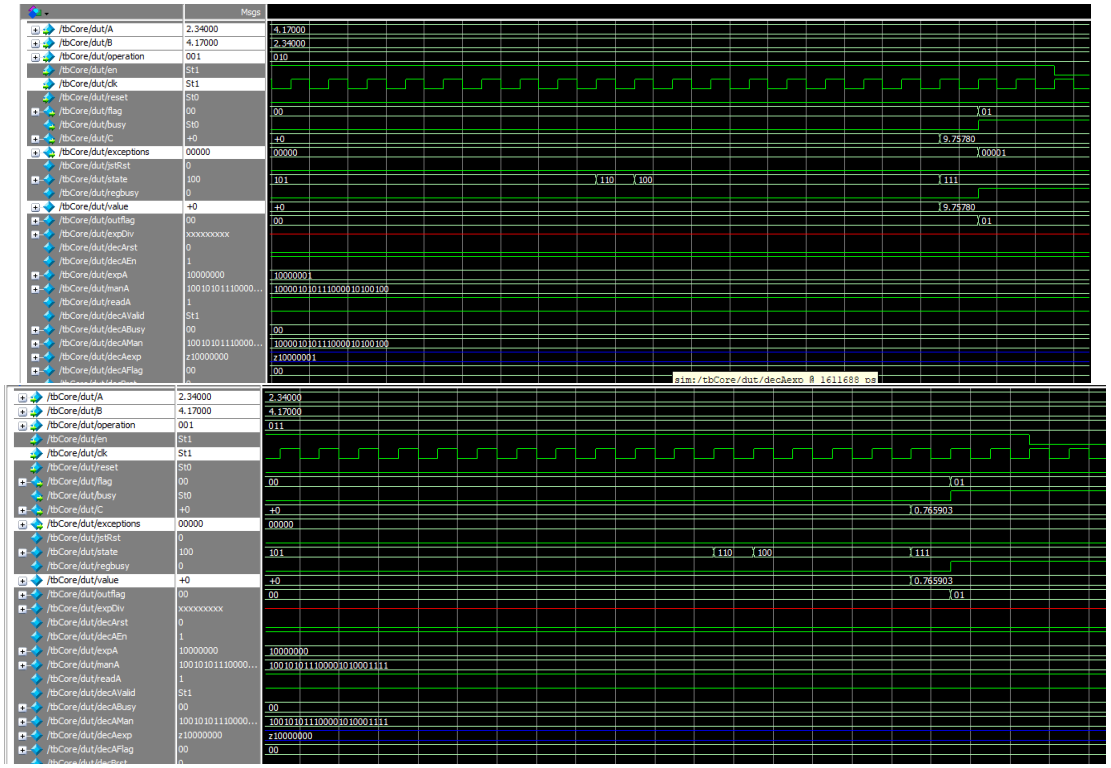

Figure 3: Multiplication and Division

3.2 Timing Analysis

Post-implementation timing analysis confirms that the design meets all specified timing constraints on the Xilinx Zynq-7000 target. The system operates reliably with a positive Worst Negative Slack (WNS), indicating sufficient timing margin.

- **System Clock:** 125 MHz (8.000 ns period).
- **Timing Closure: PASSED** (0 Failing Endpoints).
- **Setup Slack (WNS):** +1.503 ns.
- **Hold Slack (WHS):** +0.055 ns.

Check Type	Worst Slack	Total Violation	Failing Endpoints
Setup (Max Delay)	+1.503 ns	0.000 ns	0
Hold (Min Delay)	+0.055 ns	0.000 ns	0
Pulse Width	+3.500 ns	0.000 ns	0

Table 1: Post-Implementation Timing Summary

The analysis of the critical path (Logic Level = 17, Delay $\approx$ 11.542 ns) indicates that the arithmetic core is capable of operating at a maximum theoretical frequency of approximately **86.6 MHz**. The current operating frequency of 1.25 MHz is therefore well within the safe operating margin.

3.3 Resource Utilization

The design was synthesized and implemented on a Xilinx Zynq-7000 (XC7Z010-1CLG400C) FPGA. Table 2 summarizes the post-implementation resource usage.

The results demonstrate the high area efficiency of the sequential FSM-based architecture.

- **LUT Utilization (8.69%):** The Look-Up Tables are primarily consumed by the complex combinational logic in the `normalizer` (priority encoder and barrel shifter) and the 24-bit adder/subtractor logic.
- **Register Utilization (2.72%):** Flip-flop usage is minimal because the design reuses the same registers (e.g., `regMan`, `expA`) across multiple clock cycles rather than pipeline stages.
- **Efficiency:** With less than 9% of the FPGA resources occupied, this FPU leaves ample room for integrating a soft-core processor (like a MicroBlaze or RISC-V) or other peripherals on the same chip.

Resource	Used	Available	Utilization %
Slice LUTs	1530	17600	8.69 %
Slice Registers	958	35200	2.72 %
Bonded IOB	8	100	8.00 %
BUFG Ctrl	4	32	12.50 %

Table 2: Post-Implementation Resource Utilization on XC7Z010

4 Conclusion

This project successfully implemented a resource-efficient, sequential Floating Point Unit (FPU) compliant with the IEEE 754 Single Precision standard. By adopting a centralized Finite State Machine (FSM) architecture rather than a high-throughput pipeline, the design prioritized area minimization, utilizing less than 9% of the Zynq-7000 FPGA resources.

Key achievements of the implementation include:

- **Timing Closure:** The dual-clock strategy, decoupling the 125 MHz system interface from the 62.5 MHz arithmetic core, proved critical in resolving timing violations in the complex carry-chains of the iterative divider.
- **Precision & Robustness:** The FPU correctly handles all rounding modes (Round to Nearest Even) and corner cases, including Subnormal numbers, Infinity, and NaN, as verified by extensive simulation.
- **Modular Scalability:** The clear separation between the central controller and execution units allows for easy integration of additional arithmetic operations (e.g., Square Root) without disrupting the control logic.

The design demonstrates that high-precision arithmetic can be effectively implemented on low-cost FPGA fabrics by trading latency for resource efficiency, making it an ideal accelerator for embedded systems with strict area constraints.